



what about the already approximated by 9 piecewise-linear tangent of the former SoC-dependent aging weight $w(x)$. Doesnt this make the problem a full MILP?

No. The nine piecewise-linear tangents make the aging-weight function $w(x)$ itself linearizable, but your implemented degradation cost still multiplies that variable weight by a continuous discharge-energy variable. That remaining product means the model is not a full MILP when `advanced_degradation_status == 'on'` and `soc_weight_factor > 0`.

Where the nonlinearity remains

You approximate the convex aging weight

$$w(x) = 1 + 4\alpha(x - 0.5)^2, \quad x = \frac{\text{SoC}}{C},$$

using tangent inequalities such as

$$w_{m,t} \geq a_k x_{m,t-1} + b_k \quad \forall k \in \{1, \dots, 9\}.$$

That part is entirely linear. In the code, this is the section that creates `soc_w_var` and adds the tangent constraints:

```
soc_w_var = model.addVar(  
    lb=1.0,  
    ub=1.0 + soc_weight_factor,  
    name=f"soc_aging_w_{m}_{t}{tag}"  
)  
  
for x_i, w_i, slope_i in soc_weight_tangents:  
    model.addLConstr(  
        soc_w_var >= w_i + slope_i * (  
            x_m_SoC[m, t - 1] / cap - x_i  
        )  
    )
```

So far, this is a linear epigraph representation of the piecewise-linear approximation:

$$w_{m,t} = \max_k \{a_k x_{m,t-1} + b_k\}.$$

However, immediately afterwards the code forms:

```
throughput = E_neg[m, t] / discharging_efficiency
weighted_thp = throughput * soc_w
degrad_contrib = efc_base * weighted_thp / (2.0 * cap)
```

That is:

$$\left[\frac{c_m}{2C_m\eta_{\mathrm{dis}}} E^{\mathrm{neg}}_{m,t}; w_{m,t} \right]$$

Both factors are continuous decision variables:

- $E_{m,t}^{\mathrm{neg}}$: continuous discharged energy, in kWh;
- $w_{m,t}$: continuous SoC-dependent aging weight.

Thus the term

$$E_{m,t}^{\mathrm{neg}} \cdot w_{m,t}$$

is bilinear. Approximating $w(x)$ by nine tangent lines does not eliminate that product.^[1]

Why the tangent approximation alone is linear

If the cost had been only a function of SoC,

$$\text{cost}_{m,t} = g(\text{SoC}_{m,t}),$$

then your tangent formulation would indeed yield a MILP-compatible linear representation, assuming g were represented with a linear epigraph/hypograph formulation.

For example, this is linear:

$$q_{m,t} \geq a_k \frac{\text{SoC}_{m,t-1}}{C_m} + b_k \quad \forall k,$$

and then minimizing $q_{m,t}$ forces:

$$\left[\max_k \left\{ a_k \frac{\mathrm{SoC}_{m,t-1}}{C_m} + b_k \right\} \right]$$

Likewise, this would be a MILP objective:

$$\min \sum_{m,t} \gamma_{m,t} w_{m,t},$$

provided $\gamma_{m,t}$ is a constant.

But in your case $\gamma_{m,t}$ is not constant. It is proportional to the optimized V2G discharge $E_{m,t}^{\text{neg}}$.

A small numerical illustration

Suppose that at a time step:

$$E^{\text{neg}} = 100 \text{ kWh}, \quad w = 1.30.$$

The weighted throughput is:

$$100 \cdot 1.30 = 130 \text{ weighted kWh.}$$

If the optimizer changes SoC, it changes w . If it changes the V2G decision, it changes E^{neg} . The cost needs the product of those two decisions.

For two possible operating points:

State	E^{neg}	w	Weighted throughput
Mid-SoC discharge	100 kWh	1.00	100
High-/low-SoC discharge	100 kWh	1.50	150
Smaller extreme-SoC discharge	50 kWh	1.50	75

A linear model can represent the chosen SoC weight and the chosen discharge individually. It cannot calculate the last column exactly with only the continuous variables E^{neg} and w , because that column is their product.

Model classification by settings

Your model is a MILP only under specific settings:

Settings	Classification
<code>advanced_degradation_status != 'on'</code>	MILP, assuming no other nonlinear expressions are introduced elsewhere
<code>advanced_degradation_status == 'on'</code> and <code>soc_weight_factor == 0</code>	MILP, because SOC_W becomes a scalar constant rather than a variable
<code>advanced_degradation_status == 'on'</code> and <code>soc_weight_factor > 0</code>	Nonconvex MIQP / MIQCP-type model, due to $E^{\text{neg}} \times w$
Original quadratic $w(\text{SoC})$ multiplied by discharge	Higher-order nonlinear formulation, potentially cubic if represented directly

Your own comments correctly identify that the tangent representation avoids the quadratic $w(x)$ itself and avoids SOS2/binaries for that part. But the statement that this “keeps the

objective linear” is not correct for the branch where `soc_w` is `soc_w_var`, because Python/Gurobi constructs a quadratic expression at:

```
weighted_thp = throughput * soc_w
```

When `soc_weight_factor == 0`, `soc_w` is a numeric scalar, and then the expression is linear. ^[1]

How to make it a MILP

To obtain a full MILP while keeping the intended SoC-dependent throughput-aging feature, you must also linearize or discretize the **interaction** between SoC state and discharged energy.

A standard exact-with-respect-to-discretization formulation is to split discharge by SoC interval.

Let $k \in K$ index your nine SoC intervals or representative SoC levels. Introduce:

- $b_{m,t,k} \in \{0, 1\}$: SoC interval k is active;
- $e_{m,t,k} \geq 0$: V2G discharge allocated to interval k .

Use:

$$\sum_{k \in K} b_{m,t,k} = 1,$$

$$E_{m,t}^{\text{neg}} = \sum_{k \in K} e_{m,t,k},$$

$$0 \leq e_{m,t,k} \leq \bar{E}_m^{\text{neg}} b_{m,t,k}.$$

Then tie the prior-step SoC to the selected interval. If interval k spans $[\underline{x}_k, \bar{x}_k]$,

$$\underline{x}_k C_m - M(1 - b_{m,t,k}) \leq \text{SoC}_{m,t-1} \leq \bar{x}_k C_m + M(1 - b_{m,t,k}).$$

Finally, use a constant aging weight $\bar{w}_k$ or a local linear aging-cost approximation in each segment:

[
degradation_{m,t}

$\frac{c_m}{2C_m\eta_{\text{dis}}}$

$\sum_{k \in K} \bar{w}_k e_{m,t,k}$.

]

Now every term is linear:

- $\bar{w}_k$ is a constant;
- $e_{m,t,k}$ is continuous;
- the binary variables only activate ranges and capacity bounds.

The cost is no longer an exact representation of your current continuous piecewise-linear-weight-times-continuous-discharge model. It is, however, a controlled MILP approximation of the same underlying physics, and its accuracy can be improved systematically by increasing the number of SoC segments.

Important distinction

There are two separate approximations:

1. Weight-function approximation

Your current nine tangents approximate the original quadratic aging weight $w(x)$.

2. Energy-weight interaction approximation

To obtain a MILP, you must additionally approximate or discretize the product $E^{\text{neg}}w(x)$.

Your current model implements only the first approximation. The second interaction remains bilinear. ^[1]

Practical verification

You can confirm the model class from Gurobi after building it. Add this after `model_build(...)` and before optimization:

```
print("Variables:", model.NumVars)
print("Binary variables:", model.NumBinVars)
print("Linear constraints:", model.NumConstrs)
print("Quadratic constraints:", model.NumQConstrs)

quad_obj = model.getObjective()
print("Objective type:", type(quad_obj))
```

`NumQConstrs` can still be zero because your bilinearity is in the **objective**, not in an explicit quadratic constraint. The relevant diagnostic is whether `model.getObjective()` is a `gurobipy.QuadExpr`, or whether Gurobi's log reports a quadratic/nonconvex objective. Gurobi's `NonConvex` parameter governs treatment of nonconvex quadratic objectives and constraints; explicitly setting `NonConvex=2` is appropriate if you retain this formulation. ^[2] ^[3]

In short: the nine tangents linearize $w(\text{SoC})$, but not $E^{\text{neg}} \cdot w(\text{SoC})$. So with nonzero SoC weighting, the attached model remains nonlinear.

✱

1. `hdv_disposition_optimization.py`

2. <https://docs.gurobi.com/projects/optimizer/en/current/reference/parameters.html>

3. <https://docs.gurobi.com/projects/optimizer/en/current/concepts/modeling/constraints.html>